# Multi-platform distributed systems with Aggregate Computing in Kotlin: the case of proximity messaging

Luca Marchi

**ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA**
Corso di Laurea Triennale in Ingegneria e Scienze Informatiche

**Relatore**: Prof. Pianini Danilo
**Correlatore**: Dott.ssa Cortecchia Angela

Sessione III Anno Accademico 2024-2025

# Overview and Motivations

- **Context**: The world is increasingly populated by IoT and mobile devices that must communicate in dynamic environments.
- **Problem**: Centralized messaging architectures are fragile, difficult to scale, and unsuitable for pervasive and ubiquitous computing scenarios.
- **Motivation**: Resilient and more adaptive communication is needed.

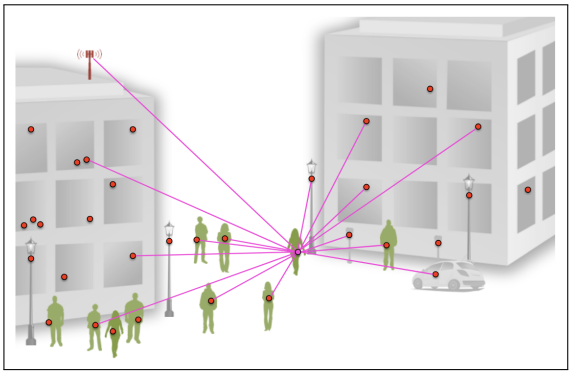

Figure: Example of a centralized system for proximity messaging.

# Goals

This thesis project has two main goals:

**Goal 1**

Design and implement a decentralized, proximity-based messaging algorithm based on the principles of Aggregate Computing.

**Goal 2**

Develop a multiplatform (Android/iOS) mobile application that uses the algorithm as core logic.

# Aggregate Computing: The paradigm

What is Aggregate Computing?

**Aggregate Computing**

Programming paradigm that treats large, interconnected systems of devices as a single, aggregate machine rather than focusing on individual components.

## Key concepts

- **Collective behavior**: The focus is on programming the group, or "aggregate," as a whole.
- **Abstraction**: It provides a higher level of abstraction that hides the complexity of managing individual devices in a vast and dynamic network.
- **Logic**: The program logic is expressed as the manipulation of data structures that extend spatially and temporally (**Field Calculus**).

# Project Specifications

## Architecture

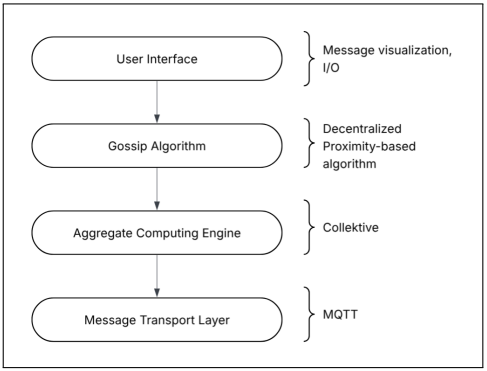


Figure: High-level overview of the architecture.

## Technologies

- **AC framework**: Collektive
- **Programming language**: Kotlin
- **Multi-platform framework**: KMP
- **GUI**: Compose Multiplatform
- **Simulator**: Alchemist Simulator
- **Transport Protocol**: MQTT

# Algorithm (Gossip-Gradient)

The algorithm is at the core of the system, allowing decentralized proximity messaging.

## Description

- **Objective**: Enable communication between nodes by finding the shortest path (**gradient**) to the message source.
- **Algorithm Family**: **Gossip-based** algorithm, which rely on local interactions between nodes to propagate information throughout the network. It uses a **Bellman-Ford-like** approach to compute the gradient.
- **Multiple Sources**: The algorithm can handle multiple message sources, allowing nodes to receive messages from different origins.
- **Parameters**: Maximum propagation distance, expiration time (TTL).

# Validation & CI

- **Simulations**: The correctness of the algorithm has been validated and tested using the Alchemist Simulator.
- **Continuous Integration**: GitHub Actions is used as CI platform to automate the build processes for both the shared codebase and the mobile application.

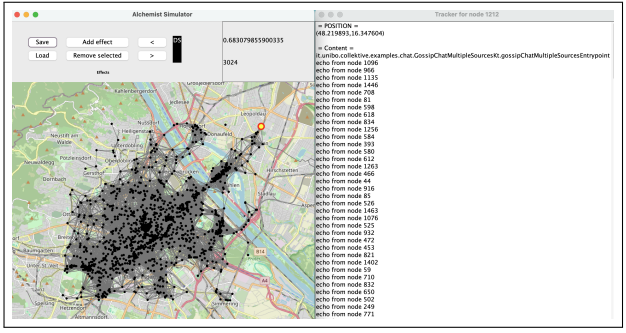


Figure: Simulation of the messaging system using real-world GPS traces. Image taken from Alchemist Simulator illustrating nodes and their communication links.

# Mobile Application (Echo)

The mobile application serves as the **GUI** of the messaging system.

## Features

- **Cross-platform**: KMP ensures compatibility with both Android and iOS devices.
- **AC runtime environment**: Collektive Engine.
- **GPS**: used to compute distances between devices.
- **MQTT**: used as transport protocol for message dissemination.

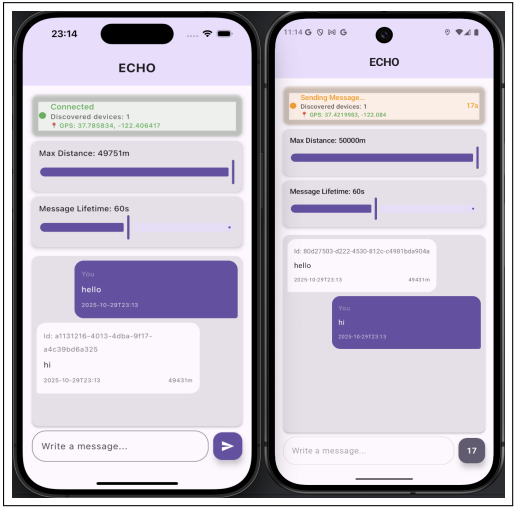


Figure: Illustration of the Echo application interface.

# Conclusions

## Final considerations

A decentralized messaging system based on the principles of Aggregate Computing has been successfully implemented.

## Future development

- **Fully decentralization**: Replace MQTT with a P2P protocol like Bluetooth or Wi-Fi Direct.
- **Deployment**: Publish Echo on official app stores to test the application in real-world scenarios.

# The End